GitHub: https://github.com/Manjunath-G-K/ShipmentIQ

ShipmentIQ

Technical Design Document

A reviewer-oriented implementation audit for a customs-broker shipment document cross-check workbench.

Live app: https://manjunath-g-k.github.io/ShipmentIQ/

Backend health: https://shipmentiq.onrender.com/api/v1/health

Release status

The public hosts are reachable, but the local CSRF bootstrap fix is not yet deployed. The live Pages bundle still uses the old cross-origin cookie reader and the live API returns 404 for /api/v1/auth/csrf. This document records that limitation explicitly.

Executive summary

ShipmentIQ helps customs-broker teams turn private invoice, packing-list, and bill-of-lading documents into structured, reviewable shipment data. It keeps source evidence beside extracted values, surfaces uncertainty and discrepancies, records human decisions, and gates provisional CSV export in the backend.

The exact problem is a manual, error-prone handoff: read several documents, re-key values, identify mismatches, resolve uncertain fields, and prepare a filing import. ShipmentIQ makes those steps explicit and auditable. It does not file customs entries, classify HSN codes, calculate duty, provide importer tracking, or certify regulatory compliance.

Verification	Result
Repository	GitHub remote verified
Frontend	GitHub Pages HTTP 200
Backend	Render health HTTP 200
Database provider	Neon documented; not publicly verifiable
CSRF bootstrap	Local only; live endpoint HTTP 404

1. Problem, users, and scope

Roles

Role	Implemented responsibilities
Documentation Executive	Sign in, create/update own shipments, upload, extract, correct, confirm/reject, validate, and request eligible CSV export.
Documentation Manager	View shipments in scope, review documents/fields, confirm/reject, validate, and view field/supplier Unchanged Extraction Rate.

The server reloads roles from PostgreSQL on protected requests. Frontend visibility is only a convenience; it is not authorization.

Implemented core build

- Shipment list, creation, and metadata update.
- Private invoice, packing-list, and bill-of-lading upload, with one document of each type per shipment.
- Synchronous OCR/text extraction and strict structured field materialization.
- Side-by-side source viewer and extracted-field review.
- Confidence, source-page/locator evidence, missing/unreadable flags, and revision history.
- Deterministic invoice-total, package, gross-weight, and net-weight validation.
- Human correction, confirmation, rejection, stale-validation handling, eligibility, and provisional CSV.
- Manager field-level and supplier-level reporting.

Scope boundary

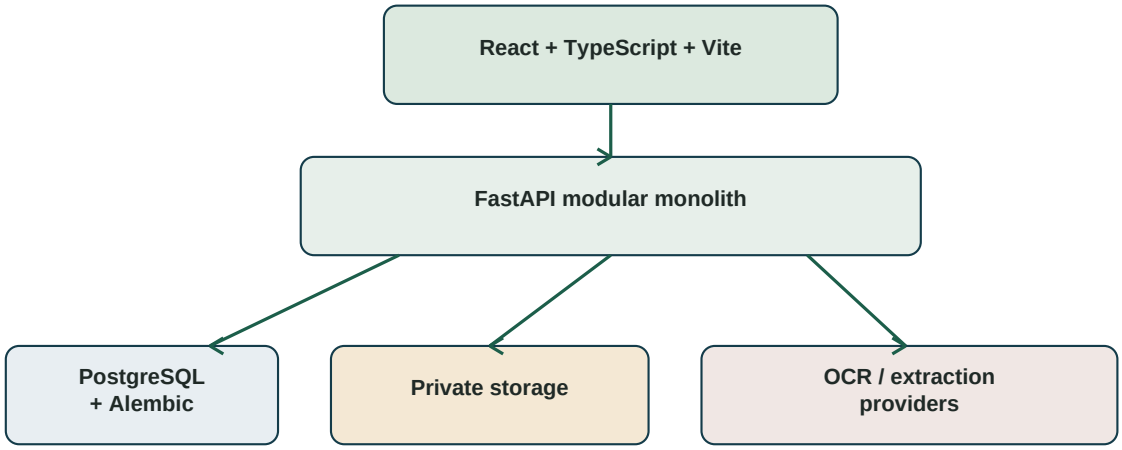
No customs-portal filing, HSN classification, duty calculation, importer tracking portal, regulatory certification, autonomous approval, or India data-residency guarantee is implemented.

2. Architecture

ShipmentIQ is a modular monolith. One FastAPI process owns authentication, authorization, transactions, document handling, extraction orchestration, review, validation, reporting, and export. This fits a small workflow product and avoids distributed infrastructure before measured scale requires it.

Technology choices

Layer	Actual implementation	Trade-off / boundary
Browser	React 18, TypeScript, Vite, plain CSS	Feature-local state; no router/state framework.
API	FastAPI, dependency injection, OpenAPI	Synchronous handlers and service transactions.
Schemas	Pydantic v2	Strict provider and request validation.
Persistence	SQLAlchemy 2, PostgreSQL, Alembic	Relational constraints and JSONB evidence.
Auth	Argon2id; short-lived JWT HttpOnly cookie	No refresh-token rotation/password recovery.
Storage	Private generated-name filesystem	Ephemeral on Render; no object store integrated.
OCR	pypdf; PyMuPDF/Pillow/Tesseract fallback	Tesseract executable is external.
Extraction	Local labelled-text; optional OpenAI-compatible	External provider setting is unverified.



Credentialed REST, exact Origin allowlist, signed CSRF header/cookie validation

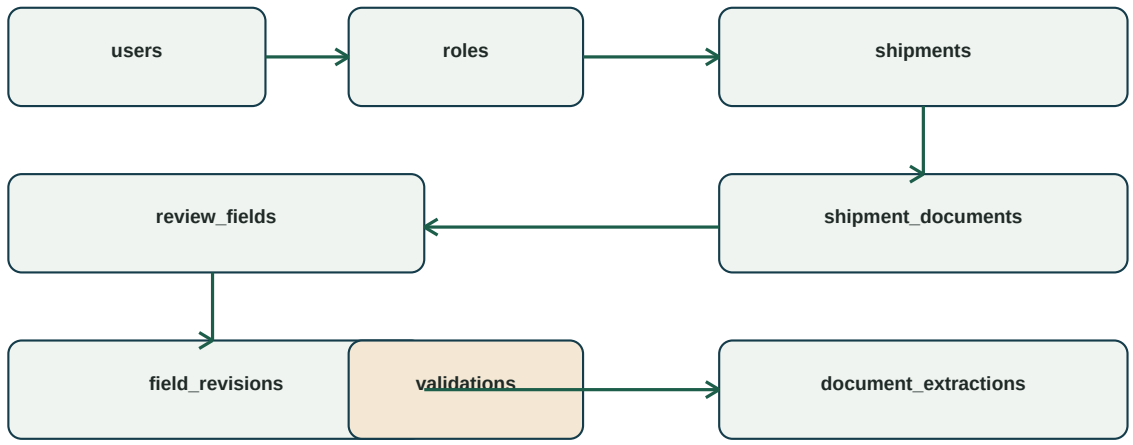
All mutations pass through authentication, role checks, input validation, exact Origin checks, and signed CSRF validation. The frontend cannot set status or export eligibility.

3. Data model

The current migrations create nine tables: users, roles, user_roles, shipments, shipment_documents, document_extractions, review_fields, field_revisions, and shipment_validations. Supplier is a string on shipments; there is no separate supplier entity.

Entity	Implemented facts
User	UUID, email, Argon2id hash, active flag, timestamps.
Role / user_roles	Two roles and many-to-many association.
Shipment	Unique reference, importer, supplier, origin, description, creator, status, timestamps.
ShipmentDocument	Shipment/uploader keys, type, filename, opaque key, content type, size, SHA-256.
DocumentExtraction	Latest run, provider names, status, page count, JSON result, safe errors.
ReviewField	Path, extracted/current values, value type, confidence, sources, status, version.
FieldRevision	Append-only action, actor, old/new values, note, timestamp.
ShipmentValidation	One result per shipment/rule, status, evidence, evaluator, timestamps.

Nine current tables; user-role association and server-side foreign keys omitted from the drawing for readability.



4. End-to-end workflows

Authentication and shipment creation

- POST /api/v1/auth/login validates credentials and exact allowed Origin, then sets a JWT cookie and signed CSRF cookie.
- The client calls /auth/me and loads shipments.
- Before writes, the corrected client calls authenticated GET /api/v1/auth/csrf, receives Cache-Control: no-store, and keeps the token in memory.
- POST /api/v1/shipments sends credentials, Origin, and X-CSRF-Token. The backend verifies the header/cookie match and signature binding to the JWT.
- The created shipment starts in DRAFT status.

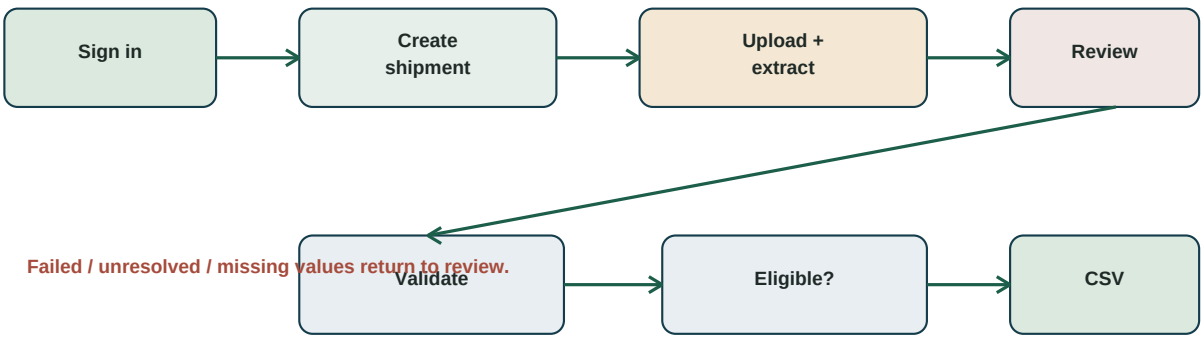
Document intake and human review

- The backend validates PDF/JPG/PNG filename, extension, size, declared type, signature, and shallow structure before storing an opaque key.
- Extraction returns ordered page records, strict fields, confidence, and page/locator evidence.
- Missing or unreadable values stay NEEDS_REVIEW. Reviewers correct scalar values with optimistic version checking, confirm, reject, and inspect history.

Validation and export

Reviewers explicitly run invoice-total, package-count, gross-weight, and net-weight checks. Fewer than two values are UNRESOLVED; different normalized values are FAILED; equal values are PASSED. Corrections and rejections make old validations STALE.

The export service recomputes eligibility inside a shipment lock. It blocks failed, unresolved, stale, missing, rejected, unconfirmed, failed-extraction, and incomplete-document conditions. Successful export streams provisional CSV and marks the shipment EXPORTED.



5. OCR and extraction design

The default setting selects LocalRuleBasedProvider. This is deterministic labelled-text parsing, not an LLM. It extracts explicitly labelled values and simple labelled/table forms from OCR text and makes no semantic-understanding or accuracy claim.

PDFs first use pypdf embedded text page by page. Pages without usable text fall back to PyMuPDF rasterization, Pillow image handling, and the pytesseract wrapper. The Tesseract executable is not installed by requirements.txt; absence becomes a safe OCR failure for scanned files.

An OpenAICompatibleProvider adapter exists behind explicit provider, URL, key, model, and timeout settings. It requests JSON-only output with confidence and sources, then validates the response strictly. Public source and live HTTP do not expose Render environment values, so no claim is made that an external LLM is configured in production.

Evidence and uncertainty

- Every populated field requires confidence in 0..1 and at least one page/locator source reference.
- Unreadable pages, missing values, malformed provider JSON, OCR failures, and provider failures become explicit statuses or safe error codes.
- The low-confidence threshold defaults to 0.80. This is an implementation assumption, not a measured quality threshold.
- No accuracy percentage is claimed because there is no labelled evaluation dataset.
- Deterministic cross-document validation consumes persisted review values and never chooses a winning document.

Human confirmation

Extraction is assistive. A discrepancy is evidence for human review, not an automatic winner. Export eligibility remains a backend policy decision.

6. UI/UX and security

Actual screens and journey

The frontend implements sign-in, shipment list, shipment creation, shipment detail/review, and manager report views. The detail view places a private document viewer beside structured fields, with upload, rerun, validation, eligibility, and export actions near the shipment heading.

Each field exposes current and extracted values, confidence, source page/locator, status, flags, correction note, actions, and history. Loading, empty, and error states are explicit. The visual system uses a sand/off-white surface, deep green navigation, restrained red/amber status accents, responsive grids, and a desktop-first detail layout.

Security controls

- Argon2id password hashing and short-lived JWT access tokens in an HttpOnly cookie.
- Signed double-submit CSRF cookie/header validation bound to the current JWT.
- Authenticated GET /auth/csrf with Cache-Control: no-store; the frontend keeps the token in memory only.
- Exact CORS allowlist, credentials enabled, explicit CSRF header, and exact Origin validation. No wildcard origins.
- Server-side role and shipment/document/field scoping.
- Private generated-name files, restrictive permissions, content checks, size limits, checksums, and path/symlink validation.
- Row locks for validation/export decisions and optimistic versions for field corrections.

Reliability limits

The initial deployment uses ephemeral local filesystem storage. Login rate limiting, refresh-token rotation, password recovery, full malware scanning, durable object storage, browser E2E tests, and operational backup/retention are not implemented.

7. Testing and verification

Check	Result	Notes
Backend pytest -q tests	39 passed, 26 skipped	Integration tests skipped because RUN_DB_INTEGRATION was not set and no isolated PostgreSQL database was provided.
Frontend npm run test	9 passed	Vitest API-client and CSRF bootstrap coverage.
Frontend npm run build	Passed	TypeScript compilation and Vite production bundle.
git diff --check	Passed	No whitespace errors.
Live frontend	HTTP 200	Pages host reachable; public bundle is pre-fix.
Live backend health	HTTP 200	Render health returned {status:ok}.
Live CSRF bootstrap	Not passed	Endpoint returned 404 because the local fix is not deployed.

Test coverage boundary

Integration tests cover PostgreSQL migrations, authentication/RBAC, shipment scoping, upload/extraction flows, review/revisions, validation state, and export eligibility. They must run only against an isolated test database, never an unknown or production database. A browser-level Playwright suite is not included.

What was not claimed

No production readiness, regulatory certification, data residency, measured 95% accuracy, measured time reduction, or amendment reduction is claimed. Client success metrics remain targets, not results.

8. Assumptions, limitations, and next work

Explicit assumptions

- 0.80 is the default low-confidence threshold because the brief supplies no threshold.
- Exact normalized numeric equality is used because no rounding/tolerance policy was supplied.
- Managers may review, decide, validate, and report; executive-only corrections, uploads, and CSV generation remain server-enforced.
- CSV field order is provisional because the filing template was not supplied.
- The documented Neon target is not independently verifiable from public runtime evidence.

Client-provided estimates and targets

The brief targets reducing data entry from 45 minutes to under 15 minutes, achieving 95% field accuracy on 200 historical shipments, and halving amended Bills of Entry within three months. These are client-provided estimates/targets. ShipmentIQ has no evaluation dataset and makes no measured claim about them.

Prioritized follow-on work

- Deploy the CSRF bootstrap fix with a coordinated Pages rebuild and authenticated browser smoke test.
- Replace ephemeral Render storage with approved private object storage and define retention/backup.
- Install and verify Tesseract in the chosen runtime and evaluate OCR on labelled documents.
- Obtain and implement the actual filing-software CSV template and field mapping.
- Add browser tests, rate limiting, malware scanning, token revocation policy, and monitoring.

Submission note: the required ZIP contains only this PDF and deployment.md. Reviewer passwords are intentionally absent pending explicit private confirmation.